

Buổi 7

Sparse table và ngăn xếp đơn điệu, bản Python

Kiến thức, mười bài tập và mười code mẫu

Ngày 23 tháng 9 năm 2026

TS. Đỗ Phúc Hảo

Tài liệu một file cho buổi 7, bản Python. Gồm bốn phần: kiến thức, mười đề bài, gợi ý và lời giải, và mười code mẫu đầy đủ.

Đề bài và bộ test dùng chung với bản C++ ở `../topics/07-sparse-stack/`. Cùng một đề, cùng một bộ test, khác ngôn ngữ. Cái khác không nằm ở thuật toán mà ở **hình dạng code mà ngôn ngữ chịu được**, và buổi này có hai chỗ đáng đo hơn cả: cách dựng một bảng $n \log n$ phần tử, và cái giá thật của việc lấy phần tử ra khỏi một list.

Buổi này cũng là buổi đầu tiên mà **cả mười bài đều chạy dưới một phần tư giây**, và điều đó không phải tình cờ: phần nặng của cả hai kỹ thuật gọi được xuống các hàm C dựng sẵn, khác hẳn segment tree của buổi 6 nơi một bài lazy ngồi ở 2,7 giây.

1. Mục lục

Phần	Nội dung
I	Kiến thức, mười bốn mục
II	Mười đề bài A tới J
III	Gợi ý phân tầng và lời giải từng bài
IV	Mười code mẫu đầy đủ

Phần I. Kiến thức

1. Hai kỹ thuật, và chỗ mỗi cái đứng

Buổi này khép lại tầng 2. Hai kỹ thuật của nó bù đúng hai chỗ mà segment tree tỏ ra thừa: khi dãy **không bao giờ thay đổi**, và khi câu hỏi thật sự là **phần tử nhỏ hơn gần nhất nằm ở đâu**.

Tình huống	Dùng gì
dãy tĩnh, hỏi tổng đoạn	tổng tiền tố, buổi 2
dãy tĩnh, hỏi cực trị hoặc gcd đoạn	sparse table, $O(1)$ mỗi truy vấn
cập nhật điểm, hỏi tổng đoạn	Fenwick, buổi 5
cập nhật đoạn hoặc phép hợp phức tạp	segment tree, buổi 6
hỏi phần tử nhỏ hơn hoặc lớn hơn gần nhất	ngăn xếp đơn điệu, $O(n)$
cực trị của cửa sổ trượt	hàng đợi đơn điệu, buổi 2

Bảng này đáng in ra dán vào sổ tay, và nó đúng với cả hai ngôn ngữ.

2. Sparse table: ý tưởng, và truy vấn $O(1)$

Dãy **không đổi**, rất nhiều truy vấn cực trị đoạn. Segment tree làm được với $O(\log n)$ mỗi truy vấn. Sparse table đổi bộ nhớ lấy tốc độ: tiền xử lý $O(n \log n)$, mỗi truy vấn **$O(1)$** .

$sp[k][i]$ giữ kết quả của khối dài 2^k bắt đầu tại i . Tầng k ghép hai khối của tầng $k-1$, lệch nhau $2^{(k-1)}$:

```
sp[k][i] = hop(sp[k-1][i], sp[k-1][i + 2^(k-1)])
```

Truy vấn đoạn $[l, r]$: lấy k là lũy thừa hai lớn nhất không vượt độ dài đoạn, rồi hợp **hai khối dài 2^k** , một khối bám mép trái và một khối bám mép phải.

Hai khối đó **chồng lên nhau** ở giữa, nên nhiều phần tử bị tính hai lần. Với phép lấy nhỏ nhất thì không sao, vì $\min(x, x) = x$. Tính chất đó gọi là **lũy đẳng**. Các phép lũy đẳng thường gặp: nhỏ nhất, lớn nhất, gcd, hoặc theo bit, và theo bit.

Phép tổng không lũy đẳng, vì $x + x$ khác x . Dùng sparse table cho tổng đoạn thì phần chồng nhau bị cộng hai lần. Với tổng thì đã có tổng tiền tố của buổi 2, vừa $O(1)$ vừa ngắn hơn nhiều. Hạn chế thứ hai: **dãy phải tĩnh**, một phép cập nhật buộc dựng lại cả bảng.

3. Dựng bảng: đừng viết hai vòng lặp lồng nhau

Đây là chỗ Python khác C++ rõ nhất trong cả buổi. Bảng có 18 tầng với $n = 2 \cdot 10^5$, tức khoảng 3,2 triệu phần tử phải tính. Bản C++ viết hai vòng lặp lồng nhau theo chỉ số và không phải trả gì thêm. Bản Python viết như vậy thì trả tiền trình thông dịch 3,2 triệu lần.

Cách viết thay thế: dựng **từng tầng một**, mỗi tầng bằng đúng một lời gọi map trên hai lát cắt của tầng dưới.

```
sp = [a]
k = 1
while (1 << k) <= n:
    prev = sp[k - 1]
    sp.append(list(map(min, prev, prev[1 << (k - 1):])))
    k += 1
```

Chi tiết đáng nhớ: map với **hai** dãy đầu vào dừng ở dãy ngắn hơn, và độ dài đó đúng bằng số vị trí bắt đầu hợp lệ của tầng mới. Không phải tính $n - 2^k + 1$ ở đâu cả.

Số đo trên bộ test lớn nhất của kho, $n = 2 \cdot 10^5$, chỉ tính phần dựng bảng:

Cách dựng	min	gcd
hai vòng lặp lồng nhau theo chỉ số	0,228 s	0,143 s
map trên hai lát cắt	0,130 s	0,054 s

Hai điều rút ra, và điều thứ hai bất ngờ hơn điều thứ nhất.

Thứ nhất, khoảng cách là 1,8 lần với min và 2,6 lần với gcd. Đó là một khoản thật nhưng **không** phải khoảng cách giữa đạt và trượt ở chủ đề này: cả hai cách đều lọt giới hạn 3 giây rất xa. Đừng nhớ nó như một mệnh lệnh sống còn, hãy nhớ nó như một thói quen.

Thứ hai, dựng bảng **gcd nhanh hơn** dựng bảng min, dù gcd là phép nặng hơn nhiều. Lý do không nằm ở thuật toán: min nhận số lượng đối số tùy ý nên mỗi lời gọi phải gói các đối số vào một tuple, còn math.gcd với hai đối số thì không. Trên ba triệu lời gọi, khoản gói tuple ấy lớn hơn cả phép Euclid.

4. bit_length thay cho logarit, và một câu của bản C++ không tái lập được

Cần k là $\text{floor}(\log_2(\text{len}))$. Trong Python nó là một phép trên số nguyên:

```
k = (r - l + 1).bit_length() - 1
```

Bản C++ tính trước một bảng lg và dặn: đừng dùng hàm logarit của số thực vì **sai làm tròn ở những giá trị đúng bằng lũy thừa hai**.

Tôi đã thử lại câu đó bằng Python và **nó không tái lập được ở cỡ dữ liệu này**. Chạy hết cả 200.000 độ dài có thể có, $\text{int}(\text{math.log}_2(x))$ khác $x.\text{bit_length}() - 1$ đúng **0 lần**; với $\text{int}(\text{math.log}(x, 2))$ cũng 0 lần; và riêng với mọi lũy thừa hai từ 2^1 tới 2^{199} cũng không sai lần nào. Chỗ math.log_2 thật sự hỏng nằm ở $x = 2^e - 1$ với e từ 49 trở lên, tức xa ngoài $n \leq 2 \cdot 10^5$.

Vậy lý do dùng bit_length ở đây là gì? Hai lý do đo được, không phải lý do làm tròn:

Cách tính, 200.000 lượt	Thời gian
$x.\text{bit_length}() - 1$	0,010 s
$\text{int}(\text{math.log}_2(x))$	0,017 s
$\text{int}(\text{math.log}(x, 2))$	0,023 s

Và lý do thứ hai không đo được nhưng quan trọng hơn: bit_length **không đi qua số thực một bước nào**, nên câu hỏi làm tròn không tồn tại chứ không phải là đã được kiểm và thấy ổn.

Cũng vì vậy Python không cần bảng lg tính trước của bản C++. Một phương thức sẵn có trên int thay cho một mảng $n + 1$ phần tử.

5. Ngăn xếp đơn điệu: list đúng là cấu trúc cần, nếu bạn chạm đúng đầu

Bài toán: với mỗi vị trí, tìm vị trí gần nhất bên trái có giá trị nhỏ hơn.

```
sti = []          # indices
stv = []          # their values, increasing from bottom to top
for i in range(n):
    x = a[i]
    while stv and stv[-1] >= x:
        stv.pop(); sti.pop()
    res[i] = sti[-1] + 1 if sti else 0
    sti.append(i); stv.append(x)
```

Ngăn xếp luôn giữ các chỉ số có giá trị tăng dần từ đáy lên đỉnh. Khi phần tử mới tới, mọi phần tử ở đỉnh không nhỏ hơn nó bị loại vĩnh viễn: chúng vừa cũ hơn vừa không nhỏ hơn nên không còn là đáp án của ai nữa. Mỗi chỉ số vào một lần và ra một lần, nên tổng chi phí là **O(n)** dù có vòng while bên trong.

Bên C++ người ta dùng vector với `push_back` và `pop_back`. Bên Python, list với `append` và `pop` là **đúng** thứ đó, và đây là một trong số ít chỗ mà cấu trúc dựng sẵn của Python rơi đúng vào hình dạng bài toán cần.

Nhưng phải chậm đúng đầu. `list.pop()` lấy ở **cuối** và là $O(1)$ khâu hao. `list.pop(0)` lấy ở **đầu** và phải dịch mọi phần tử còn lại lên một ô. Số đo, 200.000 lần lấy ra khỏi một cấu trúc 200.000 phần tử:

Phép	Thời gian
<code>list.pop()</code>	0,0061 s
<code>collections.deque.popleft()</code>	0,0048 s
<code>list.pop(0)</code>	2,3512 s

Gấp 385 lần. Và trong ngăn xếp đơn điệu thì `pop(0)` còn **sai** nữa, không chỉ chậm: nó vứt đi phần tử đáy, tức phần tử xa nhất, trong khi thứ cần vứt là phần tử đỉnh.

Đây là lý do hai stack song song ở đoạn code trên: một stack chỉ số và một stack giá trị. Tồn thêm một `append` mỗi phần tử, đổi lại bỏ được phép `a[sti[-1]]` trong vòng `while` nóng.

6. Bốn biến thể, và cách nhớ chúng

Cần tìm	Duyệt theo chiều	Điều kiện loại ở đỉnh
nhỏ hơn gần nhất bên trái	trái sang phải	đỉnh không nhỏ hơn phần tử mới
nhỏ hơn gần nhất bên phải	phải sang trái	đỉnh không nhỏ hơn phần tử mới
lớn hơn gần nhất bên trái	trái sang phải	đỉnh không lớn hơn phần tử mới
lớn hơn gần nhất bên phải	phải sang trái	đỉnh không lớn hơn phần tử mới

Cách nhớ: **chiều duyệt quyết định trái hay phải, điều kiện loại quyết định nhỏ hơn hay lớn hơn**. Thuộc một đoạn code rồi suy ra ba đoạn kia, đừng thuộc bốn đoạn.

7. Hàng đợi đơn điệu thì khác: chỗ này deque mới đúng

Ngăn xếp chỉ chậm một đầu, nên list là đủ. **Hàng đợi** đơn điệu của bài G chậm **cả hai** đầu: đẩy ra ở cuối khi có phần tử tốt hơn, và bỏ ở đầu khi phần tử đó rơi ra khỏi cửa sổ. list chỉ rở ở cuối, `collections.deque` rở ở cả hai.

Có một cách thứ ba hay được dùng: giữ list nhưng không bao giờ xóa ở đầu, chỉ cho một biến số nguyên `hx` chạy tới. Cả ba cách đều đúng. Số đo trên chính bộ test của bài G:

Cách viết	test 03, $n = 2 \cdot 10^5, k = 0$	test 09, $n = 2 \cdot 10^5$
<code>deque</code>	0,047 s	0,065 s
list với chỉ số đầu chạy tới	0,101 s	0,075 s
list với <code>pop(0)</code>	0,085 s	0,094 s

Hãy đọc kỹ dòng cuối, vì nó bác một câu mà tôi suýt viết vào tài liệu này. **`pop(0)` không hề gây TLE ở bài G**. Nó chỉ chậm hơn deque chưa tới hai lần, và cả ba cách đều cách giới hạn 3 giây rất xa.

Lý do không khó thấy một khi đã đo: với $k = 0$ và một dãy toàn giá trị bằng nhau, hai hàng đợi **không bao giờ dài quá vài phần tử**, nên `pop(0)` chẳng phải dịch gì cả. Con số 385 lần ở mục 5 là con số của một list dài 200.000, không phải con số của bài này.

Muốn thấy khoảng cách thật thì phải dựng dữ liệu ép hàng đợi dài ra: một dãy tăng dần với cửa sổ rộng, khi đó hàng đợi min giữ nguyên cả cửa sổ.

n = 50.000, k = 25.000, dãy tăng dần	Thời gian
list với pop(0)	0,166 s
deque	0,012 s

Bài học của mục này không phải là "dùng deque". Bài học là: **một tuyên bố về chi phí chỉ đúng kèm theo hình dạng dữ liệu sinh ra nó**. Bộ test hiện có của bài G không phân biệt được ba cách viết, nên nếu ai đó nộp bản pop(0) thì bộ chấm sẽ cho AC, và nó AC một cách chính đáng.

8. Kỹ thuật đóng góp

Đây là ý tưởng đáng giá nhất của buổi, dùng ở bài E và bài J.

Bài toán: tính tổng của giá trị nhỏ nhất trên **mọi** đoạn con. Số đoạn con là $2 \cdot 10^{10}$ nên không liệt kê được.

Đảo ngược cách đếm: thay vì duyệt từng đoạn rồi tìm min, hãy hỏi **mỗi phần tử là min của bao nhiêu đoạn**. Phần tử $a[i]$ là min của mọi đoạn bắt đầu trong $(left[i], i]$ và kết thúc trong $[i, right[i])$, nên số đoạn là $(i - left[i]) \cdot (right[i] - i)$, và đáp án là tổng của $a[i]$ nhân với số đó.

Mẫu này áp dụng được cho rất nhiều bài: tổng của max, tổng của hiệu, đếm số đoạn mà một phần tử là cực trị.

9. Bất đối xứng chặt và không chặt

Nếu dãy có các giá trị bằng nhau, cách đếm trên sẽ **đếm trùng**: một đoạn có hai phần tử cùng bằng min bị tính hai lần. Cách chữa chuẩn: **một bên dùng dấu chặt, bên kia dùng dấu không chặt**.

```
while stv and stv[-1] >= x:    # left: nearest STRICTLY smaller
    ...
while stv and stv[-1] > x:    # right: nearest smaller OR EQUAL
    ...
```

Như vậy trong một nhóm phần tử bằng nhau, chỉ **phần tử trái nhất** được tính là min của đoạn chứa cả nhóm, và mỗi đoạn được đếm đúng một lần.

Đây là loại lỗi rất khó thấy: với dữ liệu ngẫu nhiên trên khoảng giá trị rộng thì hai phần tử hiếm khi bằng nhau, nên bài vẫn đúng cho tới khi gặp test có nhiều giá trị lặp. Đề bài E và J cố tình giới hạn giá trị trong 1 tới 10^4 trong khi n lên $2 \cdot 10^5$, để giá trị lặp xuất hiện dày đặc.

10. Bài J: một mẹo Python xóa hẳn cái bẫy chép nhầm

Bản C++ giải bài J bằng một hàm nhận thêm một cờ want_min và đảo chiều mọi phép so sánh theo cờ đó. Nó chạy được, nhưng nó giữ nguyên chỗ hỏng: bất đối xứng ở mục 9 phải đúng ở **hai** bộ so sánh, và chép nhầm một trong hai thì bài sai đúng ở những dãy có giá trị lặp.

Trong Python có cách bỏ hẳn bộ thứ hai. Tổng các **cực đại** của một dãy bằng **âm** tổng các **cực tiểu** của dãy đã đổi dấu:

```
tong_max = -tong_min([-x for x in a])
```

Nên chỉ còn **một** hàm, với **một** cặp dấu so sánh cố định. Không có chỗ thứ hai để chép nhầm vào.

Một lời tự hỏi đáng đặt: đổi dấu cả dãy có làm hỏng tính bất đối xứng không? Không, vì phép đổi dấu biến các giá trị bằng nhau thành các giá trị bằng nhau, nên quy tắc "phần tử trái nhất của nhóm sở hữu đoạn" giữ nguyên. Và cái giá là một list comprehension trên $2 \cdot 10^5$ phần tử, rẻ hơn hẳn việc mang một cờ want_min vào tận vòng while trong cùng, nơi nó sẽ bị kiểm một lần cho mỗi lần pop suốt cả lượt chạy.

11. Hình chữ nhật lớn nhất, và lính canh

Bài D. Với mỗi cột, hình chữ nhật cao bằng cột đó kéo dài sang trái tới cột thấp hơn đầu tiên, và sang phải tới cột thấp hơn đầu tiên. Một lượt ngăn xếp cho cả hai biên: khi cột i đẩy cột j ra khỏi ngăn xếp, i chính là cột thấp hơn đầu tiên bên phải của j , còn đỉnh mới của ngăn xếp là cột thấp hơn đầu tiên bên trái.

Hai chi tiết quyết định:

- **Lính canh ở cuối.** Trong Python cách gọn nhất là `h.append(-1)` rồi chạy vòng tới n thay vì $n - 1$. Nhờ đó mọi cột còn nằm trong ngăn xếp đều được xử lý mà không cần viết thêm vòng lặp dọn dẹp.
- **Bề rộng là $i - \text{traí} - 1$, không phải $i - \text{traí}$.** Lệnh một đơn vị ở đây cho ra diện tích **lớn hơn** thực tế, và con số lớn hơn thì trông vẫn hợp lý.

Bài F là bài D áp cho từng hàng: $h[j]$ là số ô 1 liên tiếp kết thúc tại hàng hiện tại ở cột j , gặp số 0 thì **đặt lại về 0**.

Một chỗ Python phải khác. Bản C++ viết bài F bằng một hàm `largest_rect` gọi n lần. Ở đây hàm ấy được gỡ thành mã thẳng trong vòng lặp hàng, vì $n \cdot m$ chạm 10^6 và một lời gọi hàm Python được tính tiền lúc chạy. Cùng lý do, phần cập nhật h không phải một vòng theo chỉ số mà một list comprehension trên `zip(h, hang)`, để vòng lặp nằm trong C.

12. Đọc dữ liệu, và một mẹo riêng của bài F

Vẫn là dòng quan trọng nhất của mọi buổi trong nhánh này:

```
data = sys.stdin.buffer.read().split()
```

Riêng bài F có thêm một chỗ tiết kiệm được. Ma trận có tới 10^6 ô, nhưng câu hỏi duy nhất về mỗi ô là "nó có phải số 1 không". Vậy đừng `int()` cả triệu token, hãy so thẳng với `b'1'`:

```
mot = b'1'
h = [c + 1 if v == mot else 0 for c, v in zip(h, hang)]
```

`data` là một list các đối tượng bytes, nên về phải bắt buộc có tiền tố `b`. So `v == '1'` sẽ luôn sai và cả ma trận thành số 0, một lỗi im lặng cho ra đáp án 0 chứ không báo gì.

13. Cạm bẫy của buổi này

Chín cái đầu là cạm bẫy thuật toán, đúng như bản C++. Ba cái cuối chỉ có ở Python.

- **Dùng sparse table cho tổng.** Hai khối chồng nhau làm phần giữa bị cộng hai lần.
- **Truy vấn chỉ lấy một khối.** Chỉ đúng khi độ dài đoạn tính cỡ bằng một lũy thừa hai, nên test mẫu ngắn vẫn có thể qua.
- **Ghép hai khối lệch nhau 2^k thay vì $2^{(k-1)}$** khi dựng tầng mới.
- **Sai dấu so sánh khi đẩy ngăn xếp,** làm các giá trị bằng nhau bị xử lý sai.
- **Dùng cùng một dấu ở cả hai lượt** trong kỹ thuật đóng góp, gây đếm trùng.
- **Lệch một đơn vị khi tính bề rộng.** Là $i - \text{traí} - 1$, không phải $i - \text{traí}$.
- **Quên lính canh ở cuối** bài hình chữ nhật, bỏ sót các cột còn nằm trong ngăn xếp.
- **Quên đặt lại chiều cao về 0** khi gặp số 0 ở bài ma trận.
- **Chỉ cộng 1 thay vì $r - l + 1$** ở bài đếm đoạn con.
- **`pop(0)` thay cho `pop()`** trong ngăn xếp. Sai đầu, và ở một list dài thì chậm 385 lần.
- **List dùng làm hàng đợi hai đầu.** Chỗ đó cần deque. Nhưng xem mục 7: khoảng cách thật phụ thuộc dữ liệu, đừng nói nó gây TLE khi chưa đo trên chính bộ test đang dùng.

- **So một token bytes với một chuỗi str.** Luôn cho False, và không có lỗi nào được ném ra.

Cạm bẫy **không** có ở Python: tràn số. Bản C++ phải dùng long long cho diện tích $2 \cdot 10^{14}$, cho số đoạn con $2 \cdot 10^{10}$, cho lượng nước $2 \cdot 10^{14}$. Số nguyên Python không tràn, nên cả ba chỗ ấy mất hẳn cái bẫy. Đó cũng là lý do bảng bẫy của chủ đề này không có mục nào về kiểu dữ liệu.

14. Bộ khung tự hỏi khi gặp đề mới

- Dãy có thay đổi không? Nếu không, cân nhắc sparse table hoặc tổng tiền tố trước khi nghĩ tới segment tree.
- Phép toán có lũy đẳng không? Nếu không thì sparse table loại.
- Đề có hỏi về phần tử gần nhất thoả mãn một bất đẳng thức không? Nghĩ ngăn xếp đơn điệu.
- Đề có hỏi tổng trên **mọi** đoạn con không? Nghĩ kỹ thuật đóng góp.
- Dữ liệu có nhiều giá trị bằng nhau không? Nếu có, kiểm lại dấu chặt và không chặt.
- Cấu trúc của tôi chạm một đầu hay hai đầu? Một đầu thì list, hai đầu thì deque.
- Vòng lặp trong cùng chạy bao nhiêu lượt? Trên 10^6 lượt thì hãy hỏi tiếp: có hàm dựng sẵn nào làm được việc này không.

Kế tiếp: sang **Phần II** làm bài. Làm xong hãy tự chấm trước khi mở **Phần III**.

Phần II. Mười bài tập

Đề PDF	Ở đây	Kỹ thuật chính	Cạm bẫy bản Python	File nộp
A	Bài 1	sparse table, min tĩnh	dựng bảng bằng vòng lặp chỉ số	bai01.py
B	Bài 2	sparse table, gcd tĩnh	lệch offset khi cắt lát	bai02.py
C	Bài 3	ngăn xếp đơn điệu	pop(0) thay cho pop()	bai03.py
D	Bài 4	hình chữ nhật trong biểu đồ	lính canh và bề rộng	bai04.py
E	Bài 5	kỹ thuật đóng góp	hai lượt cùng dấu	bai05.py
F	Bài 6	hình chữ nhật trong ma trận	int() cả triệu ô, so bytes với str	bai06.py
G	Bài 7	cửa sổ trượt, hai hàng đợi	list làm hàng đợi hai đầu	bai07.py
H	Bài 8	hứng nước mưa	bề ngang vùng chứa nước	bai08.py
I	Bài 9	lớn hơn tiếp theo trên vòng tròn	dấu so sánh không chặt	bai09.py
J	Bài 10	đóng góp hai lần	chép hàm thành hai bản	bai10.py

Giới hạn thời gian **3 giây** cho mọi bài, gấp ba bản C++. Con số ấy đặt theo số đo chứ không theo một hệ số: bài chậm nhất của chủ đề này là bài A với 232 ms, tức 8 phần trăm giới hạn. Tự chấm:

```
python3 chay.py topics/07-sparse-stack/solutions/bai01.py
python3 chay.py topics/07-sparse-stack --tat-ca
```

Bài A. Giá trị nhỏ nhất trên đoạn, dãy tĩnh

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên **không thay đổi** và q truy vấn. Mỗi truy vấn cho hai chỉ số l và r , hãy cho biết giá trị nhỏ nhất trong đoạn đó.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)
- q dòng tiếp theo, mỗi dòng l và r ($1 \leq l \leq r \leq n$)

Kết quả

q dòng, mỗi dòng một số.

Ví dụ

Dữ liệu vào	Kết quả
5 3	2
5 2 8 1 9	1
1 3	1
2 5	
4 4	

Giải thích

Đoạn 1 tới 3 của dãy 5, 2, 8, 1, 9 có giá trị nhỏ nhất là 2. Đoạn 2 tới 5 có nhỏ nhất là 1. Đoạn chỉ gồm vị trí 4 là chính giá trị 1.

Lưu ý

Bài này giải được bằng segment tree, nhưng vì dãy không đổi nên có cách trả lời mỗi truy vấn trong $O(1)$. Hãy tìm cách đó. Khi dựng bảng, hãy hỏi thêm một câu riêng của Python: 3,2 triệu lượt vòng lặp có cách nào đẩy xuống một hàm dựng sẵn không?

Bài B. Ước chung lớn nhất trên đoạn, dãy tĩnh

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên dương **không thay đổi** và q truy vấn. Mỗi truy vấn cho l và r , hãy cho biết ước chung lớn nhất của các phần tử trong đoạn.

Dữ liệu vào

- Dòng 1: n và q ($1 \leq n, q \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)
- q dòng tiếp theo, mỗi dòng l và r

Kết quả

q dòng, mỗi dòng một số.

Ví dụ

Dữ liệu vào	Kết quả
5 3	6
12 18 24 6 9	3
1 3	3
4 5	
1 5	

Giải thích

Ước chung lớn nhất của 12, 18, 24 là 6, của 6 và 9 là 3, và của cả năm số cũng là 3.

Lưu ý

Dùng đúng cấu trúc của bài A. Nhưng trước đó hãy tự trả lời: cấu trúc ấy làm hai khối chồng lên nhau, vậy phép nào chịu được điều đó và phép nào thì không? Và nếu bạn dựng bảng bằng lát cắt, hãy đếm lại thật kỹ xem hai lát cắt lệch nhau bao nhiêu.

Bài C. Phần tử nhỏ hơn gần nhất bên trái

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên. Với mỗi vị trí i , hãy cho biết chỉ số j lớn nhất thoả mãn $j < i$ và $a_j < a_i$. Nếu không có, in 0 .

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^9$)

Kết quả

Một dòng gồm n số.

Ví dụ

Dữ liệu vào	Kết quả
6 2 5 3 7 1 4	0 1 1 3 0 5

Giải thích

Với $i = 4$ giá trị là 7, phần tử nhỏ hơn gần nhất bên trái là $a_3 = 3$ nên đáp án là 3. Với $i = 5$ giá trị là 1, không có phần tử nào bên trái nhỏ hơn nó nên đáp án là 0.

Lưu ý

Dãy có thể có nhiều giá trị bằng nhau, và đề hỏi **nhỏ hơn thật sự**. Hãy thử một dãy mà mọi phần tử giống hệt nhau. Về phía Python: `list` đúng là cấu trúc cần ở đây, nhưng hãy chắc rằng bạn lấy phần tử ra ở **cuối**.

Bài D. Hình chữ nhật lớn nhất trong biểu đồ cột

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho biểu đồ gồm n cột, cột thứ i cao h_i và rộng 1. Hãy tìm diện tích lớn nhất của một hình chữ nhật nằm trọn trong biểu đồ.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên h_i ($0 \leq h_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
6 2 1 5 6 2 3	10

Giải thích

Hai cột cao 5 và 6 cho hình chữ nhật cao 5 rộng 2, diện tích 10. Không có cách nào đạt hơn.

Lưu ý

Khi tính bề rộng, hãy đếm lại thật cẩn thận xem nó là hiệu hai chỉ số hay hiệu đó trừ một, và đừng quên những cột vẫn còn nằm trong ngăn xếp lúc vòng lặp kết thúc. Diện tích đạt $2 \cdot 10^{14}$, nhưng trong Python đó không phải một vấn đề.

Bài E. Tổng của giá trị nhỏ nhất trên mọi đoạn con

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên dương. Xét toàn bộ các đoạn con liên tiếp không rỗng, hãy tính tổng của giá trị nhỏ nhất trên mỗi đoạn.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^4$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 1 2 4	17

Giải thích

Mười đoạn con cho các giá trị nhỏ nhất là 3, 1, 1, 1, 1, 1, 1, 2, 2, 4 và tổng bằng 17.

Lưu ý

Số đoạn con là $2 \cdot 10^{10}$ nên không thể liệt kê. Hãy đảo ngược cách đếm: mỗi phần tử là giá trị nhỏ nhất của bao nhiêu đoạn? Và chú ý giá trị a_i chỉ tới 10^4 trong khi n tới $2 \cdot 10^5$, tức là sẽ có rất nhiều giá trị lặp.

Bài F. Hình chữ nhật toàn số 1 lớn nhất

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho ma trận n dòng và m cột, mỗi ô là 0 hoặc 1. Hãy tìm diện tích lớn nhất của một hình chữ nhật chỉ gồm các ô mang giá trị 1.

Dữ liệu vào

- Dòng 1: n và m ($1 \leq n, m \leq 1000, n \cdot m \leq 10^6$)
- n dòng tiếp theo, mỗi dòng m số 0 hoặc 1

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
<pre>3 4 1 0 1 0 1 0 1 1 1 1 1 1</pre>	4

Giải thích

Hình chữ nhật toàn số 1 lớn nhất có diện tích 4. Một cách đạt được là hai cột cuối ở hai hàng dưới, tức khối 2×2 ; hàng cuối cùng gồm bốn số 1 cũng cho diện tích 4.

Lưu ý

Bài này là bài D lặp lại theo từng hàng. Hãy nghĩ xem mỗi hàng cho bạn biểu đồ nào, và điều gì phải xảy ra khi gặp một ô mang giá trị 0. Đây cũng là bài có nhiều token nhất cả buổi, 10^6 ô, nên hãy hỏi xem mỗi ô có thật sự cần một lời gọi `int()` không.

Bài G. Đếm đoạn con có độ chênh lệch nhỏ

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên và một số k . Hãy đếm số đoạn con liên tiếp không rỗng có hiệu giữa giá trị lớn nhất và giá trị nhỏ nhất không vượt quá k .

Dữ liệu vào

- Dòng 1: n và k ($1 \leq n \leq 2 \cdot 10^5, 0 \leq k \leq 2 \cdot 10^9$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
5 2 1 3 2 5 4	9

Giải thích

Năm đoạn độ dài 1, cộng thêm 1 3, 1 3 2, 3 2 và 5 4, tổng cộng 9 đoạn.

Lưu ý

Khi cửa sổ đã hợp lệ, hãy đếm cho đủ chứ đừng chỉ đếm một đoạn: đáp án có thể đạt $2 \cdot 10^{10}$. Cấu trúc bạn cần ở đây chạm vào **cả hai** đầu, khác hẳn ngăn xếp của bài C, nên hãy chọn lại kiểu dữ liệu cho đúng.

Bài H. Hứng nước mưa

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho n cột nước, cột thứ i cao h_i và rộng 1. Trời mưa và nước đọng lại giữa các cột. Hãy tính tổng lượng nước hứng được.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên h_i ($0 \leq h_i \leq 10^9$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
12 0 1 0 2 1 0 1 3 2 1 2 1	6

Giải thích

Nước chỉ đọng ở chỗ có cột cao hơn ở cả hai phía. Cộng lại các vũng nhỏ trong địa hình này được đúng 6 đơn vị nước.

Lưu ý

Có cách dùng tiền tố và hậu tố cực đại, và trong Python cách ấy còn hấp dẫn hơn vì `itertools.accumulate` dựng cả hai mảng ở tốc độ C . Nhưng cách dùng ngăn xếp đơn điệu mới là cách buổi này muốn bạn tập. Khi tính bề ngang một vũng, hãy nhớ hai bức tường không chứa nước.

Bài I. Phần tử lớn hơn tiếp theo trên vòng tròn

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên xếp thành vòng tròn, tức là sau phần tử cuối là phần tử đầu. Với mỗi vị trí i , hãy cho biết **giá trị** của phần tử đầu tiên lớn hơn a_i khi đi theo chiều tăng chỉ số vòng quanh, hoặc -1 nếu không có.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($|a_i| \leq 10^9$)

Kết quả

Một dòng gồm n số.

Ví dụ

Dữ liệu vào	Kết quả
3 1 2 1	2 -1 2

Giải thích

Phần tử đầu tiên lớn hơn 1 khi đi từ vị trí 1 là $a_2 = 2$. Từ vị trí 2 đi vòng cả vòng không gặp số nào lớn hơn 2. Từ vị trí 3 đi vòng qua vị trí 1 rồi tới vị trí 2.

Lưu ý

Đề hỏi **lớn hơn thật sự**, nên phần tử bằng nhau không tính. Hãy thử một dãy mà mọi phần tử giống hệt nhau. Đi vòng hai lượt là đủ, nhưng lượt thứ hai có được đẩy chỉ số mới vào ngăn xếp không?

Bài J. Tổng của độ chênh lệch trên mọi đoạn con

Giới hạn thời gian: 3 giây | Giới hạn bộ nhớ: 256 MB | Vào ra chuẩn

Cho dãy n số nguyên dương. Xét toàn bộ các đoạn con liên tiếp không rỗng, hãy tính tổng của hiệu giữa giá trị lớn nhất và giá trị nhỏ nhất trên mỗi đoạn.

Dữ liệu vào

- Dòng 1: n ($1 \leq n \leq 2 \cdot 10^5$)
- Dòng 2: n số nguyên a_i ($1 \leq a_i \leq 10^4$)

Kết quả

Một số nguyên duy nhất.

Ví dụ

Dữ liệu vào	Kết quả
4 3 1 2 4	13

Giải thích

Tổng của các giá trị lớn nhất trên mọi đoạn con là 30, tổng của các giá trị nhỏ nhất là 17, hiệu bằng 13.

Lưu ý

Bài này là bài E làm hai lần. Nếu bạn viết hai hàm riêng thì rất dễ chép nhầm và làm hỏng đúng cái chi tiết khiến bài E không đếm trùng. Trong Python có một cách viết chỉ cần **một** hàm, hãy thử nghĩ ra nó trước khi mở Phần III.

Phần III. Gợi ý và lời giải

Đọc từng tầng một, dừng lại ngay khi đã nghĩ ra. Code đầy đủ nằm ở Phần IV.

Bài	Cái bẫy thật sự	Riêng với Python
A	truy vấn chỉ lấy một khối	dựng bảng bằng vòng lặp chỉ số, chậm 1,8 lần
B	tưởng sparse table dùng được cho mọi phép toán	lát cắt lệch 2^k thay vì $2^{(k-1)}$
C	sai dấu so sánh nên giá trị bằng nhau bị giữ lại	<code>pop(0)</code> sai đầu, và chậm 385 lần
D	lệch một đơn vị khi tính bề rộng, và quên lính canh	không tràn số nên bẫy long long biến mất
E	hai lượt quét dùng cùng một dấu nên đếm trùng	không có
F	gấp số 0 mà không đặt lại chiều cao cột	<code>int()</code> một triệu lần, và hàm con gọi 1000 lần
G	mỗi đầu phải phải đếm mọi đầu trái	<code>list</code> chỉ rẽ ở một đầu, cần <code>deque</code>
H	lệch một đơn vị khi tính bề ngang vùng chứa nước	<code>accumulate</code> cảm dỗ nhưng lạc đề
I	dùng dấu không chặt nên phần tử bằng nhau cũng tính	<code>step % n</code> đắt hơn một phép so sánh
J	cộng hai tổng thay vì trừ	đổi dấu dãy để chỉ cần một hàm

Bài A. Giá trị nhỏ nhất trên đoạn, dãy tĩnh

Gợi ý 1. Dãy không đổi. Điều đó cho phép tiền xử lý nặng một lần rồi trả lời rất nhanh.

Gợi ý 2. Nếu bạn có sẵn giá trị nhỏ nhất của mọi khối có độ dài là lũy thừa hai, thì một đoạn bất kỳ phủ được bằng bao nhiêu khối như thế?

Gợi ý 3. Đúng hai khối, một bám mép trái và một bám mép phải. Chúng chồng lên nhau ở giữa, nhưng với phép lấy nhỏ nhất thì chồng bao nhiêu cũng không sao.

Lời giải. Sparse table, tiền xử lý $O(n \log n)$, mỗi truy vấn $O(1)$. Lấy $k = (r - l + 1).bit_length() - 1$ rồi hợp `sp[k][l]` với `sp[k][r - 2k + 1]`.

Chỗ mất điểm thứ nhất: chỉ lấy một khối. Chỉ đúng khi độ dài đoạn tình cờ là một lũy thừa hai, nên test mẫu ngắn vẫn có thể qua.

Chỗ mất điểm thứ hai, riêng của Python: dựng bảng bằng hai vòng lặp lồng nhau. Xem mục I.3. Dựng từng tầng bằng map trên hai lát cắt đo được 0,130 s, hai vòng lặp theo chỉ số đo được 0,228 s. Cả hai đều lọt giới hạn 3 giây, nên đây là chuyện thói quen chứ không phải chuyện sống chết. Nhưng thói quen ấy là thứ cứu bạn ở một bài khác nặng hơn.

Không phải lo. Bản C++ dựng sẵn một bảng lg để khởi gọi logarit số thực. Python có `int.bit_length()` sẵn, nên cả mảng ấy biến mất. Và xem mục I.4: lý do dùng nó là nhanh hơn và không đi qua số thực, **không** phải vì logarit số thực làm tròn sai ở cỡ dữ liệu này. Tôi đã kiểm cả 200.000 độ dài, lệch 0 lần.

Bài B. Ước chung lớn nhất trên đoạn, dãy tĩnh

Gợi ý 1. Dùng đúng cấu trúc của bài A, chỉ đổi phép hợp.

Gợi ý 2. Nhưng trước đó hãy tự hỏi: hai khối chồng nhau nghĩa là một số phần tử bị tính hai lần. Phép nào chịu được điều đó?

Gợi ý 3. Phép chịu được gọi là **luỹ đẳng**: hợp một phần tử với chính nó cho lại nó.

Lời giải. Thay min bằng `math.gcd`, không đổi gì khác. Nhỏ nhất, lớn nhất, gcd, hoặc theo bit, và theo bit đều lũy đẳng. **Phép tổng thì không**, vì $x + x$ khác x ; với tổng đã có tổng tiền tố của buổi 2, vừa $O(1)$ vừa ngắn hơn nhiều. Đó là lý do sparse table gần như chỉ dùng cho cực trị và gcd.

Cái bẫy riêng của cách viết bằng lát cắt. Tầng k ghép hai khối của tầng $k-1$ lệch nhau 2^{k-1} , tức `prev[1 << (k - 1):]`, không phải `prev[1 << k:]`. Viết nhầm thành 2^k thì tầng mới vừa sai giá trị vừa **ngắn hơn cần thiết**, nên lỗi hiện ra ở một truy vấn chạm gần cuối dãy dưới dạng `IndexError` chứ không phải dưới dạng một con số sai. Đây là loại lỗi mà cách viết bằng vòng lặp chỉ số không mắc phải, và nó là cái giá phải trả cho 1,8 lần tốc độ.

Python. `math.gcd` là hàm C, nhanh hơn hẳn một hàm Euclid tự viết. Một con số ngược đời đáng nhớ: dựng bảng gcd đo được 0,054 s trong khi dựng bảng min đo được 0,130 s, dù gcd nặng hơn min rất nhiều. Lý do là min nhận số đối số tùy ý nên phải gói chúng vào tuple ở mỗi lời gọi, còn gcd hai đối số thì không.

Bài C. Phần tử nhỏ hơn gần nhất bên trái

Gợi ý 1. Nếu có hai vị trí $j < i$ với $a_j \geq a_i$, thì a_j có bao giờ còn là đáp án của một vị trí nào phía sau i không?

Gợi ý 2. Không bao giờ, vì a_i vừa nhỏ hơn hoặc bằng lại vừa gần hơn. Vậy hãy loại a_j ngay.

Gợi ý 3. Cái còn lại là một dãy tăng dần, giữ trong ngăn xếp. Mỗi chỉ số vào một lần và ra một lần nên tổng chi phí là $O(n)$.

Lời giải. Duyệt từ trái sang phải. Trước khi đẩy i vào, loại khỏi đỉnh mọi phần tử **không nhỏ hơn** $a[i]$; cái còn lại trên đỉnh chính là đáp án.

Dấu so sánh là cái bẫy. Đề hỏi nhỏ hơn thật sự, nên phần tử bằng nhau phải bị loại. Viết $>$ thay vì $\geq$ thì với dữ liệu có giá trị lặp bạn trả về một vị trí có giá trị bằng chứ không nhỏ hơn. Test 02 của bài này gồm bốn số bằng nhau chính là để bắt lỗi đó.

Python. list với `append` và `pop` **đúng** là ngăn xếp cần dùng, và đây là một trong ít chỗ cấu trúc dựng sẵn rơi đúng vào hình dạng bài toán. Nhưng `pop()` lấy ở cuối còn `pop(0)` lấy ở đầu. Dùng nhầm thì vừa **sai** ý nghĩa, vừa chậm: 200.000 lần lấy ra khỏi một list 200.000 phần tử tốn 0,0061 s với `pop()` và 2,3512 s với `pop(0)`, tức 385 lần.

Một chi tiết nhỏ nữa: giữ **hai** list song song, một cho chỉ số và một cho giá trị. Tốn thêm một `append` mỗi phần tử, đổi lại bỏ được phép `a[sti[-1]]` trong vòng `while`.

Bài D. Hình chữ nhật lớn nhất trong biểu đồ cột

Gợi ý 1. Với mỗi cột, hình chữ nhật cao đúng bằng cột đó kéo dài được tới đâu?

Gợi ý 2. Sang trái tới cột thấp hơn đầu tiên, sang phải tới cột thấp hơn đầu tiên. Đó chính là bài C, làm cả hai phía.

Gợi ý 3. Một lượt ngăn xếp cho cả hai biên: khi cột i đẩy cột j ra, i là biên phải của j còn đỉnh mới của ngăn xếp là biên trái.

Lời giải. Duyệt một lượt, mỗi lần một cột bị đẩy ra thì tính ngay diện tích hình chữ nhật cao bằng nó.

Chi tiết thứ nhất: lính canh ở cuối. Trong Python cách gọn nhất là `h.append(-1)` rồi cho vòng lặp chạy tới n . Nhờ đó mọi cột còn nằm trong ngăn xếp đều được xử lý mà không cần viết thêm vòng lặp dọn dẹp, và cũng không cần nhớ viết nó.

Chi tiết thứ hai: bề rộng là $i - \text{tra}i - 1$, không phải $i - \text{tra}i$. Lệch một đơn vị ở đây cho ra diện tích lớn hơn thực tế, và một con số lớn hơn thì trông vẫn hợp lý.

Không phải lo. Diện tích đạt $2 \cdot 10^{14}$, và bên C++ đó là lý do phải dùng `long long`. Số nguyên Python không tràn, nên bài này mất đúng cái bẫy ấy. Cái còn lại là giá của vòng lặp, nên thân vòng viết thẳng: không

hàm con, không gọi max, chỉ một phép so sánh.

Bài E. Tổng của giá trị nhỏ nhất trên mọi đoạn con

Gợi ý 1. Số đoạn con là $2 \cdot 10^{10}$, không liệt kê được. Hãy đảo ngược cách đếm.

Gợi ý 2. Thay vì duyệt từng đoạn rồi tìm min, hãy hỏi mỗi phần tử là min của bao nhiêu đoạn.

Gợi ý 3. Phần tử $a[i]$ là min của mọi đoạn bắt đầu sau vị trí nhỏ hơn gần nhất bên trái và kết thúc trước vị trí nhỏ hơn gần nhất bên phải.

Lời giải. Hai lượt ngăn xếp cho left và right, rồi cộng $a[i] * (i - left[i]) * (right[i] - i)$. Đây là **kỹ thuật đóng góp**, dùng được cho rất nhiều bài.

Cái bẫy là các giá trị bằng nhau. Nếu cả hai lượt quét dùng cùng một dấu so sánh thì một đoạn có hai phần tử cùng bằng min sẽ được đếm hai lần. Cách chữa chuẩn là **bất đối xứng**: lượt trái tìm phần tử nhỏ hơn **thật sự**, lượt phải tìm phần tử nhỏ hơn **hoặc bằng**. Khi đó trong một nhóm phần tử bằng nhau, chỉ phần tử trái nhất được tính là min của đoạn chứa cả nhóm.

Lỗi này rất khó thấy vì với dữ liệu ngẫu nhiên trên khoảng giá trị rộng thì hai phần tử hiếm khi bằng nhau. Test cổ tối đa cố tình giới hạn giá trị trong 1 tới 10^4 để các giá trị lặp xuất hiện dày đặc, và test 02 gồm ba số bằng nhau thì bắt lỗi ngay.

Python. Bài này không có bẫy riêng nào của ngôn ngữ. Nó nằm ở đây để bạn thấy điều ngược lại cũng có thật: không phải bài nào cũng cần một mẹo Python, và đi tìm một mẹo ở chỗ không có là cách tự làm code mình khó đọc.

Bài F. Hình chữ nhật toàn số 1 lớn nhất

Gợi ý 1. Quét từng hàng. Với hàng hiện tại, hãy nghĩ tới một biểu đồ cột.

Gợi ý 2. $h[j]$ là số ô 1 liên tiếp kết thúc tại hàng này ở cột j . Khi đó đáp án cho hàng này chính là bài D.

Lời giải. Cập nhật h theo từng hàng rồi chạy bài D trên nó. Độ phức tạp $O(n \cdot m)$.

Đòng đặt lại về 0 là toàn bộ cái bẫy. Quên nó thì cột cứ tăng mãi qua các ô 0, và bạn tính ra những hình chữ nhật không tồn tại. Kết quả lớn hơn thực tế, nên nó không trông như một lỗi.

Python, ba chỗ. Thứ nhất, $n \cdot m$ chạm 10^6 , nên cập nhật h phải là một list comprehension trên $zip(h, \text{hang})$ chứ không phải một vòng theo chỉ số. Thứ hai, đừng `int()` cả triệu token khi câu hỏi duy nhất là "ô này có phải số 1 không": so thẳng token với `b'1'`. Chú ý tiền tố `b`, vì data chứa các đối tượng bytes; so với `'1'` thì mọi ô thành 0 và chương trình in ra 0 mà không báo lỗi gì. Thứ ba, bản C++ gói phần biểu đồ thành hàm `largest_rect` gọi n lần; ở đây hàm ấy phải được gỡ thành mã thẳng, vì một lời gọi hàm Python được tính tiền lúc chạy và đây là vòng nóng nhất của cả buổi.

Kết quả sau ba chỗ đó: 238 ms trên ma trận 1000×1000 .

Bài G. Đếm đoạn con có độ chênh lệch nhỏ

Gợi ý 1. Thu hẹp một đoạn thì hiệu giữa max và min không thể tăng. Tính chất đó là đơn điệu, nên cửa sổ trượt dùng được.

Gợi ý 2. Cửa sổ cần biết cả max lẫn min của nó. Cấu trúc nào cho bạn cực trị của một cửa sổ đang trượt?

Gợi ý 3. Hai hàng đợi đơn điệu, đúng cấu trúc của buổi 2, một cái giảm dần và một cái tăng dần.

Lời giải. Với mỗi đầu phải r , đẩy đầu trái l sang phải tới khi cửa sổ hợp lệ. Khi đó **mọi** đầu trái từ l tới r đều cho một đoạn hợp lệ, nên $\text{ans} += r - l + 1$.

Chỗ mất điểm. Chỉ cộng 1 thì bạn chỉ đếm đoạn dài nhất kết thúc tại r , bỏ sót mọi đoạn con của nó. Đáp án đạt $2 \cdot 10^{10}$; bên C++ đó còn là một bẫy tràn số, bên Python thì không, nên chỉ còn lại lỗi đếm thiếu.

Python, và một tuyên bố tôi đã phải rút lại. Hàng đợi chậm cả hai đầu nên `collections.deque` là cấu trúc đúng, khác với ngăn xếp của bài C. Nhưng tôi suýt viết rằng `list.pop(0)` gây TLE ở bài này, và số đo bác bỏ: trên test 03, bản `pop(0)` đo 0,085 s so với 0,047 s của bản `deque`; trên test 09 là 0,094 s so với 0,065 s. Lý do là với $k = 0$ và dãy toàn giá trị bằng nhau thì hai hàng đợi không bao giờ dài quá vài phần tử.

Khoảng cách chỉ mở ra khi hàng đợi thật sự dài. Dựng riêng một dãy tăng dần với $n = 50.000$ và $k = 25.000$, hàng đợi min giữ nguyên cả cửa sổ: `pop(0)` đo 0,166 s, `deque` đo 0,012 s.

Đây là mục đáng nhớ nhất của cả Phần III, và nó không nói về `deque`. Nó nói rằng **một tuyên bố về chi phí chỉ đúng kèm theo hình dạng dữ liệu sinh ra nó**. Bộ test hiện có không phân biệt nổi ba cách viết, nên bản `pop(0)` sẽ được AC, và nó AC một cách chính đáng.

Bài H. Hứng nước mưa

Gợi ý 1. Lượng nước trên đầu cột i bằng bao nhiêu?

Gợi ý 2. Bằng giá trị nhỏ hơn trong hai cột cao nhất ở hai phía, trừ đi chiều cao của chính cột i .

Gợi ý 3. Có cách dùng tiền tố và hậu tố cực đại. Nhưng cũng có cách dùng ngăn xếp đơn điệu, và đó là cách buổi này muốn bạn tập.

Lời giải với ngăn xếp. Giữ các chỉ số có chiều cao giảm dần. Khi một cột cao hơn tới, nó đóng lại một lòng chảo: lấy đáy chảo ra, tường trái là đỉnh ngăn xếp mới, tường phải là cột vừa tới, và lượng nước là bề ngang nhân chiều sâu.

Hai chỉ tiết: **bề ngang là $i - j - 1$** chứ không phải $i - j$, vì hai bức tường không chứa nước; và **phải thoát khi ngăn xếp rỗng** sau lúc lấy đáy ra, vì khi đó không có tường trái nên nước chảy hết ra ngoài.

Python. Cách tiền tố và hậu tố cực đại ở đây kém dễ hơn bình thường, vì `itertools.accumulate(h, max)` dựng cả mảng tiền tố ở tốc độ O và bài giải xong trong ba dòng. Nó đúng, nó nhanh, và nó **lạc đề**: kỹ thuật buổi này muốn luyện là ngăn xếp. Hãy viết bản ngăn xếp trước, rồi viết bản `accumulate` để so, đó mới là cách học được cả hai.

Bài I. Phần tử lớn hơn tiếp theo trên vòng tròn

Gợi ý 1. Trên dãy thẳng, bài này là ngăn xếp đơn điệu quen thuộc.

Gợi ý 2. Với vòng tròn, đi qua dãy hai lần là đủ, vì sau $2n$ bước mọi phần tử đã nhìn hết vòng.

Gợi ý 3. Nhưng ở lượt thứ hai thì không được đẩy chỉ số mới vào ngăn xếp nữa.

Lời giải. Chạy `step` từ 0 tới $2n - 1$, lấy i là vị trí tương ứng, đẩy ra khỏi ngăn xếp mọi chỉ số có giá trị nhỏ hơn $a[i]$ và trả lời chúng bằng $a[i]$. Chỉ lượt đầu mới được đẩy chỉ số mới vào.

Dấu so sánh phải chặt. Đề hỏi lớn hơn thật sự, nên dùng dấu không chặt là bạn trả lời bằng một phần tử bằng nhau. Test 02 gồm bốn số bằng nhau, đáp án đúng là bốn số -1 , còn bản sai trả về chính giá trị đó.

Một ghi chú thẳng thắn, chép lại từ bản C++ và nó vẫn đúng ở đây. Cái bẫy tác giả bản C++ định dạy ở bài này lúc đầu là "đẩy chỉ số vào ngăn xếp cả ở lượt thứ hai". Cổng kiểm báo đột biến đó sống sót, và đối chiếu 3000 bộ dữ liệu cho thấy hai bản **tương đương về ngữ nghĩa**. Đã thay bằng một lỗi thật, là dấu so sánh. Bài học lặp lại ở mục I.7 của tài liệu này với `pop(0)`: đột biến sống sót không tự động nghĩa là bộ test yếu.

Python. `step % n` trên $4 \cdot 10^5$ lượt là một phép chia lấy dư thật mỗi lượt; `step - n if step >= n` else `step` làm đúng việc ấy rẻ hơn. Và như bài C, giữ một list giá trị song song để bỏ phép `a[sti[-1]]` khỏi vòng `while`.

Bài J. Tổng của độ chênh lệch trên mọi đoạn con

Gợi ý 1. Tổng của hiệu bằng hiệu của hai tổng.

Gợi ý 2. Vậy chỉ cần làm bài E hai lần, một lần cho min và một lần cho max.

Gợi ý 3. Nhưng hãy tìm cách **không** phải viết bài E hai lần.

Lời giải. Bản C++ viết một hàm nhận thêm cờ `want_min` rồi đảo chiều mọi phép so sánh theo cờ đó. Cách ấy chạy được nhưng vẫn để lại hai bộ dấu so sánh, tức hai chỗ có thể chép nhầm tính bất đối xứng của bài E.

Trong Python có cách bỏ hẳn bộ thứ hai: tổng các cực đại của a bằng **âm** tổng các cực tiểu của $-a$.

```
tong_max = -tong_min([-x for x in a])
ket_qua = tong_max - tong_min(a)
```

Chỉ còn **một** hàm với **một** cặp dấu cố định. Phép đổi dấu biến các giá trị bằng nhau thành các giá trị bằng nhau, nên quy tắc "phần tử trái nhất của nhóm sở hữu đoạn" giữ nguyên và bất đối xứng vẫn đúng.

Cái bẫy còn lại nằm ở đúng một ký tự: hai nửa phải **trừ** nhau. Cộng chúng lại cho một con số lớn hơn, và với dữ liệu dương thì nó vẫn trông như một tổng hợp lý.

Python. Cái giá của mẹo này là một list comprehension trên $2 \cdot 10^5$ phần tử, rẻ hơn hẳn việc mang một cờ `want_min` vào tận vòng `while` trong cùng, nơi nó bị kiểm một lần cho mỗi lần `pop` suốt cả lượt chạy.

Phần IV. Mười code mẫu đầy đủ

Mười file này là **đúng** nội dung trong solutions/ref/, tức đúng thứ bộ chấm chạy. Công tools/qa-note-code.py so từng dòng và trượt nếu hai bên lệch nhau, vì mã in ra giấy không được bộ test nào chấm nên nó là chỗ dễ lạc hậu nhất trong cả kho.

Bài A. Giá trị nhỏ nhất trên đoạn, dãy tĩnh

```
# Problem A: static range minimum, answered in O(1) by a sparse table.
# sp[k][i] holds the minimum of the block of length 2^k starting at i. A query is covered
# by TWO blocks of the same length that overlap in the middle, which is legal only
# because minimum is idempotent: min(x, x) = x. Sum would be wrong here.
# The trap this file aims at is taking ONE block instead of two: that is right only when
# the length of the segment happens to be an exact power of two.
#
# Python notes.
# 1. The table is 18 levels of up to 2*10^5 entries, so two nested Python loops over
# indices pay interpreter overhead about 3.2 million times. Here each LEVEL is
# built by one map() call over two slices of the level below, so the iteration runs
# in C. Measured on this repo's largest test (n = 2*10^5), build only:
#   nested index loops ..... 0.228 s
#   map() over two slices ..... 0.130 s    <- this file
#   map with two iterables stops at the shorter one, which is exactly the number of
#   valid starting positions of the new level, so no length arithmetic is needed.
# 2. floor(log2(len)) is (len).bit_length() - 1: integers only, no float anywhere.
#   Measured over all 200000 possible lengths: bit_length 0.010 s against 0.017 s for
#   int(math.log2). The C++ note about a float logarithm ROUNDING wrong does not
#   reproduce at this size: over those same 200000 lengths int(math.log2(x)) and
#   x.bit_length() - 1 disagreed 0 times. The reason to prefer bit_length here is
#   speed and the absence of a float, not a rounding bug.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    a = list(map(int, data[2:2 + n]))

    sp = [a]
    k = 1
    while (1 << k) <= n:
        prev = sp[k - 1]
        sp.append(list(map(min, prev, prev[1 << (k - 1):])))
        k += 1

    qs = list(map(int, data[2 + n:2 + n + 2 * q]))
    out = []
    ghi = out.append
    for j in range(0, 2 * q, 2):
        l = qs[j] - 1
        r = qs[j + 1] - 1
        k = (r - l + 1).bit_length() - 1    # floor(log2(len)), no float logarithm
        row = sp[k]
        x = row[l]; y = row[r - (1 << k) + 1]
        ghi(x if x < y else y)

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()
```

Bài B. Ước chung lớn nhất trên đoạn, dãy tĩnh

```
# Problem B: static range gcd with a sparse table.
# Same skeleton as problem A, one name different: min becomes gcd. It works for the same
# reason: gcd(x, x) = x, so the two overlapping blocks may count the middle twice.
# Sum is NOT idempotent, which is why a sparse table can never answer range sums.
# The trap this file aims at is the offset between the two halves of a level: level k
# glues two blocks of length 2^(k-1), so the shift is 2^(k-1) and not 2^k.
#
# Python note. math.gcd is a C function, and it is called about 3.2 million times while
# building, so how the loop around it is written matters. Measured on this repo's
# largest test (n = 2*10^5), build only:
#   nested index loops with math.gcd ..... 0.143 s
#   map(gcd, level, level[shift:]) ..... 0.054 s      <- this file
# Note this build is FASTER than the same build with min in problem A (0.130 s), even
# though gcd is the heavier operation: min takes a variable number of arguments and has
# to pack them into a tuple on every call, while gcd of two arguments does not.
import sys
from math import gcd

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); q = int(data[1])
    a = list(map(int, data[2:2 + n]))

    sp = [a]
    k = 1
    while (1 << k) <= n:
        prev = sp[k - 1]
        sp.append(list(map(gcd, prev, prev[1 << (k - 1):])))
        k += 1

    qs = list(map(int, data[2 + n:2 + n + 2 * q]))
    out = []
    ghi = out.append
    for j in range(0, 2 * q, 2):
        l = qs[j] - 1
        r = qs[j + 1] - 1
        k = (r - l + 1).bit_length() - 1
        row = sp[k]
        ghi(gcd(row[l], row[r - (1 << k) + 1]))

    sys.stdout.write('\n'.join(map(str, out)) + '\n')

main()
```

Bài C. Phần tử nhỏ hơn gần nhất bên trái

```
# Problem C: for every position, the nearest position on its left holding a SMALLER
# value. This is the canonical monotonic stack: it keeps indices whose values increase
# from bottom to top. Before pushing i, pop everything that is not smaller than a[i];
# whatever is left on top is exactly the answer. Every index is pushed once and popped
# once, so the whole sweep is O(n) even though there is a while inside the for.
# The trap is the comparison sign: the statement asks for STRICTLY smaller, so equal
# values must be popped too, which is why the test with four equal numbers exists.
```

```
#
# Python notes.
# 1. A Python list IS the right stack: append() and pop() work at the END and are O(1)
#    amortised. pop(0) is a different operation, it takes from the FRONT, so it is
#    wrong here AND it shifts every remaining element. Measured, 200000 removals from
#    a list of 200000: pop() 0.006 s, pop(0) 2.351 s.
# 2. Two parallel stacks, one of indices and one of values, cost one append more per
#    element but save an a[st[-1]] double indexing inside the hot while loop.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    sti = []          # indices
    stv = []          # their values, strictly increasing from bottom to top
    res = [0] * n
    for i in range(n):
        x = a[i]
        while stv and stv[-1] >= x:
            stv.pop(); sti.pop()
        if sti:
            res[i] = sti[-1] + 1
            sti.append(i); stv.append(x)

    sys.stdout.write(' '.join(map(str, res)) + '\n')

main()
```

Bài D. Hình chữ nhật lớn nhất trong biểu đồ cột

```
# Problem D: largest rectangle in a histogram.
# For each bar, the widest rectangle of that height reaches left until a smaller bar and
# right until a smaller bar. One monotonic stack finds both boundaries in a single pass:
# when bar i pops bar j, i is the first smaller bar to the right of j and the new stack
# top is the first smaller bar to its left.
# Two details decide the answer. A SENTINEL bar of height -1 appended at the end flushes
# whatever is still on the stack, so no clean up loop is needed. And the width is
# i - left - 1, not i - left: an off by one here gives a LARGER area, and a larger
# number still looks plausible.
# Complexity O(n).
#
# Python note. The C++ version needs long long because the area reaches 2*10^14. Python
# integers do not overflow, so that particular trap simply does not exist here. What
# does exist is the cost of the loop itself, so the body is written flat: no helper
# function, no max() call, just a comparison.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    h = list(map(int, data[1:1 + n]))
    h.append(-1)          # sentinel, flushes the stack at the end

    sti = []
    stv = []
```

```

best = 0
for i in range(n + 1):
    cur = h[i]
    while stv and stv[-1] >= cur:
        cao = stv.pop(); sti.pop()
        trai = sti[-1] if sti else -1
        dien_tich = cao * (i - trai - 1)
        if dien_tich > best:
            best = dien_tich
    sti.append(i); stv.append(cur)

sys.stdout.write(str(best) + '\n')

main()

```

Bài E. Tổng của giá trị nhỏ nhất trên mọi đoạn con

```

# Problem E: sum of the minimum over every contiguous subarray.
# There are up to  $2 \cdot 10^{10}$  subarrays, so they cannot be listed. Reverse the counting:
# instead of walking subarrays and taking a minimum, ask how many subarrays each element
# is the minimum of. Element  $i$  is the minimum of exactly  $(i - \text{left}) * (\text{right} - i)$  of
# them, where left is the nearest STRICTLY smaller element on the left and right is the
# nearest element on the right that is smaller OR EQUAL.
# That asymmetry between the two sweeps is the whole trap. With the same sign on both
# sides, a subarray whose minimum appears twice is counted twice; with the asymmetry,
# only the LEFTMOST of a group of equal values owns the subarray containing the group.
# Complexity  $O(n)$ .
#
# Python note. The bug hides on random data over a wide range, because two values are
# rarely equal there. It is the test with repeated values that exposes it, which is also
# why the statement caps  $a_i$  at  $10^4$  while  $n$  reaches  $2 \cdot 10^5$ .
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    left = [0] * n
    right = [0] * n
    sti = []; stv = []
    for i in range(n):
        x = a[i]
        while stv and stv[-1] >= x:          # nearest STRICTLY smaller on the left
            stv.pop(); sti.pop()
        left[i] = sti[-1] if sti else -1
        sti.append(i); stv.append(x)

    del sti[:]; del stv[:]
    for i in range(n - 1, -1, -1):
        x = a[i]
        while stv and stv[-1] > x:          # nearest smaller OR EQUAL on the right
            stv.pop(); sti.pop()
        right[i] = sti[-1] if sti else n
        sti.append(i); stv.append(x)

    total = 0
    for i in range(n):

```

```

        total += a[i] * (i - left[i]) * (right[i] - i)

    sys.stdout.write(str(total) + '\n')

main()

```

Bài F. Hình chữ nhật toàn số 1 lớn nhất

```

# Problem F: largest all ones rectangle in a binary matrix.
# Sweep the rows. For the current row, h[j] is how many consecutive ones end at this row
# in column j, and the answer for the row is the largest rectangle in that histogram,
# which is problem D. Total cost O(n*m).
# The trap is the single word "reset": a zero must put the column back to 0. Forget it
# and the columns keep growing through the zeros, producing rectangles that do not
# exist, and the answer comes out too large.
#
# Python notes.
# 1. n*m reaches 10^6, so the row update must not be a per cell index loop. One list
# comprehension over zip(h, row) does the same work with the iteration in C.
# 2. The tokens are compared as BYTES against b'1' instead of being converted with
# int(): one million int() calls are pure waste when the only question is whether
# the cell is a one.
# 3. The histogram sweep is written inline, not as a helper called n times, because a
# Python function call is charged at run time and here it would be charged 1000
# times around a loop that is already the hot part of the program.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); m = int(data[1])
    p = 2

    h = [0] * m
    best = 0
    mot = b'1'
    for _ in range(n):
        hang = data[p:p + m]; p += m
        h = [c + 1 if v == mot else 0 for c, v in zip(h, hang)]

        sti = []; stv = []
        for i, cur in enumerate(h):
            while stv and stv[-1] >= cur:
                cao = stv.pop(); sti.pop()
                trai = sti[-1] if sti else -1
                dien_tich = cao * (i - trai - 1)
                if dien_tich > best:
                    best = dien_tich
            sti.append(i); stv.append(cur)
        # The sentinel of problem D, written as a flush so the row list is not copied.
        while stv:
            cao = stv.pop(); sti.pop()
            trai = sti[-1] if sti else -1
            dien_tich = cao * (m - trai - 1)
            if dien_tich > best:
                best = dien_tich

    sys.stdout.write(str(best) + '\n')

```

```
main()
```

Bài G. Đếm đoạn con có độ chênh lệch nhỏ

```
# Problem G: count subarrays whose maximum minus minimum is at most k.
# Shrinking a window can only shrink the spread, so the property is monotone and a
# sliding window works. The window has to know its own maximum and its own minimum, and
# that is exactly two monotonic queues, the structure of session 2: one holding
# decreasing values and one holding increasing values.
# The trap is the counting step. Once the window ending at r is as long as it can be,
# EVERY left end from l to r gives a valid subarray, so the answer grows by r - l + 1
# and not by 1. Adding 1 counts only the longest subarray ending at r.
# Complexity O(n).
#
# Python note. A monotonic QUEUE, unlike the stack of problem C, is touched at both
# ends, and a Python list is cheap only at the END. collections.deque is cheap at both.
# Measured, 200000 removals from a structure of 200000 elements:
#   list.pop() ..... 0.006 s      deque.popleft() ... 0.005 s
#   list.pop(0) ..... 2.351 s
# On the tests shipped with this problem the difference does not decide anything, and
# that is worth saying out loud: with k = 0 the queues never hold more than a couple of
# entries, so a pop(0) version measures 0.085 s against 0.047 s for this one. The gap
# only opens when the queues stay long, for instance an increasing array with a wide
# window: n = 50000, k = 25000 gives 0.166 s with pop(0) against 0.012 s with a deque.
import sys
from collections import deque

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0]); k = int(data[1])
    a = list(map(int, data[2:2 + n]))

    mxi = deque(); mxv = deque()      # indices and values, values decreasing
    mni = deque(); mnv = deque()      # indices and values, values increasing
    ans = 0
    l = 0
    for r in range(n):
        x = a[r]
        while mxv and mxv[-1] <= x:
            mxv.pop(); mxi.pop()
        mxv.append(x); mxi.append(r)
        while mnv and mnv[-1] >= x:
            mnv.pop(); mni.pop()
        mnv.append(x); mni.append(r)

        while mxv[0] - mnv[0] > k:
            if mxi[0] == l:
                mxi.popleft(); mxv.popleft()
            if mni[0] == l:
                mni.popleft(); mnv.popleft()
            l += 1
        ans += r - l + 1      # every left end from l to r works, up to 2e10 of them

    sys.stdout.write(str(ans) + '\n')

main()
```

Bài H. Hứng nước mưa

```
# Problem H: trapped rain water, solved with a monotonic stack.
# The stack keeps indices whose heights decrease from bottom to top. When a taller bar
# arrives it closes a basin: pop the bottom of the basin, the right wall is the arriving
# bar and the left wall is the new stack top, and the slab of water between them is
# width times depth.
# Two details. The width is i - leftWall - 1, not i - leftWall, because the two walls
# themselves hold no water. And when the stack becomes empty after the pop there is no
# left wall at all, so the water spills out and the loop must stop.
# Complexity O(n).
#
# Python note. There is also a prefix and suffix maximum solution, and in Python it is
# tempting because both prefix arrays can be built with itertools.accumulate at C speed.
# The stack version is the one worth practising, and it is the one the session teaches,
# so this file uses it. The same two parallel stacks as problem C avoid indexing h twice
# inside the while.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    h = list(map(int, data[1:1 + n]))

    sti = []; stv = []      # indices and heights, heights decreasing bottom to top
    nuoc = 0
    for i in range(n):
        hi = h[i]
        while stv and stv[-1] < hi:
            day = stv.pop(); sti.pop()
            if not sti:
                break      # no left wall, the water spills out
            j = sti[-1]
            tuong = stv[-1]
            cao = tuong if tuong < hi else hi
            nuoc += (i - j - 1) * (cao - day)
        sti.append(i); stv.append(hi)

    sys.stdout.write(str(nuoc) + '\n')

main()
```

Bài I. Phần tử lớn hơn tiếp theo trên vòng tròn

```
# Problem I: next greater element on a circle.
# Walking the array twice is enough: after 2n steps every position has seen the whole
# circle. The stack holds indices whose values decrease, and an arriving value pops and
# answers every index it beats. On the second pass no new index may be pushed, otherwise
# an element would be allowed to answer itself from behind.
# The trap is the comparison sign: the statement asks for STRICTLY greater, so a value
# equal to the one on top of the stack must NOT pop it. The test with four equal numbers
# expects four answers of -1, while the non strict version returns the value itself.
# Complexity O(n).
#
# Python note. step % n on 4*10^5 iterations is a real modulo each time; a comparison
```

```

# and a subtraction do the same job for less. Keeping a parallel stack of values also
# removes one a[st[-1]] indexing from the inner while.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    ans = [-1] * n
    sti = []; stv = []
    for step in range(2 * n):
        i = step - n if step >= n else step
        x = a[i]
        while stv and stv[-1] < x:
            ans[sti.pop()] = x
            stv.pop()
        if step < n:
            sti.append(i); stv.append(x)

    sys.stdout.write(' '.join(map(str, ans)) + '\n')

main()

```

Bài J. Tổng của độ chênh lệch trên mọi đoạn con

```

# Problem J: sum over every subarray of (maximum minus minimum).
# The sum of a difference is the difference of two sums, so the answer is the sum of
# maxima minus the sum of minima, and each half is problem E.
# The C++ version writes one routine with a flag that flips every comparison. This file
# does something cheaper and safer: the sum of MAXIMA of a equals minus the sum of
# MINIMA of the negated array. So there is exactly ONE routine, with one fixed pair of
# comparison signs, and the asymmetry that stops equal values being counted twice cannot
# be copied wrong into a second place, because there is no second place.
# The trap left in the open is the last line: the two halves are SUBTRACTED.
# Complexity O(n).
#
# Python note. Negating the array is one list comprehension over 2*10^5 elements, far
# cheaper than carrying a want_min flag into the innermost while, where it would be
# tested once per pop for the whole run.
import sys

def main():
    data = sys.stdin.buffer.read().split()
    n = int(data[0])
    a = list(map(int, data[1:1 + n]))

    def tong_min(b):
        left = [0] * n
        right = [0] * n
        sti = []; stv = []
        for i in range(n):
            x = b[i]
            while stv and stv[-1] >= x:      # nearest STRICTLY smaller on the left
                stv.pop(); sti.pop()
            left[i] = sti[-1] if sti else -1
            sti.append(i); stv.append(x)

```

```
del sti[:]; del stv[:]
for i in range(n - 1, -1, -1):
    x = b[i]
    while stv and stv[-1] > x:      # nearest smaller OR EQUAL on the right
        stv.pop(); sti.pop()
    right[i] = sti[-1] if sti else n
    sti.append(i); stv.append(x)

s = 0
for i in range(n):
    s += b[i] * (i - left[i]) * (right[i] - i)
return s

tong_max = -tong_min([-x for x in a])
sys.stdout.write(str(tong_max - tong_min(a)) + '\n')

main()
```